

Plan Your Computer Networks Preparation

Follow data from an application message to a remote service and back. Learn addresses, ports, routing, DNS, TCP/UDP, TLS, and HTTP through local experiments. Keep transport guarantees distinct from application guarantees: successful delivery of bytes does not prove a business operation completed once.

For: Software engineers who need to understand request paths, transport behavior, naming, routing, and network failures rather than memorize protocol acronyms.

Suggested pace: 6 weeks with local networking labs | Beginner to intermediate

What you will be able to do

- Explain where naming, routing, transport, security, and application semantics fit.
- Distinguish connection failure, timeout, reset, and application-level rejection.
- Reason about bandwidth, latency, flow control, congestion, and connection reuse.
- Use authorized packet or request inspection to test a network hypothesis.

Build the right mental model

Layers answer different questions

IP routes packets; transport protocols connect endpoints or deliver datagrams; TLS protects a channel; HTTP defines request semantics. A DNS name, IP address, port, and URL are related but not interchangeable. Debug the boundary that actually failed.

TCP is a byte stream

TCP provides ordered reliable bytes within a connection, not application message boundaries. Reads can return partial data or combine writes. Applications need framing, limits, and handling for disconnects while a message is incomplete.

Latency is not bandwidth

A fast link can still have high round-trip delay. Connection setup, DNS, TLS, server queueing, and payload transfer contribute differently. Flow control protects a receiver; congestion control responds to network capacity. Neither removes the need for bounded application queues.

Study with evidence.

Use a local client and server you control. Predict a request trace, introduce one safe delay or disconnect, and compare the result. Capture only authorized traffic and avoid storing credentials, private payloads, or other people's communications.

Week 1: Addressing and delivery

Study IP addresses, subnets, routes, ports, sockets, and local versus remote destinations. Inspect your own machine's interfaces and routes. Draw the path to a local service and explain which component chooses the next hop.

Deliverable and check: A labeled path diagram and a client that connects to an explicitly chosen local endpoint.

Week 2: Transport semantics

Implement a framed TCP exchange and compare it with a UDP datagram example. Test partial reads, disconnects, and timeouts. Learn handshakes, retransmission, ordering, flow control, and congestion at a conceptual level.

Deliverable and check: A parser that waits for a complete bounded message and does not assume one read equals one write.

Weeks 3-4: Web protocols

Study DNS resolution, TLS identity checks, HTTP methods, status codes, caching, and connection reuse. Compare browser timing with a command-line HTTP client. Learn that HTTP/3 uses QUIC rather than TCP.

Deliverable and check: A request timeline separating DNS, connection, TLS, server time, and response transfer.

Weeks 5-6: Failure reasoning

Practice slow dependencies, retry storms, proxy timeouts, resets, and DNS changes. Build an end-to-end deadline and retry policy for a read-only request, then explain why a write needs idempotency or reconciliation.

Deliverable and check: A troubleshooting checklist based on evidence and a diagram showing where each timeout is enforced.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Estimate in-flight data and test framing

A connection has a 100 Mb/s path and a 40 ms round-trip time. Separately, a local server sends a length-prefixed message in several writes. Use the examples to distinguish throughput constraints from application parsing.

A worked approach

1. Convert units first: 100 million bits/second times 0.04 seconds is 4 million bits, about 500,000 bytes. This bandwidth-delay product estimates the data needed in flight to fill that idealized path, not guaranteed application throughput.
2. Send a four-byte length followed by a payload from your local server, splitting the payload across writes. The client must accumulate the header, validate a maximum length, and then collect exactly the declared payload bytes.
3. Disconnect the server before the payload completes. The client should return an incomplete-message error, not process truncated data or allocate unbounded memory. Repeat with two messages sent back-to-back.

Expected result

You can explain why available bandwidth, RTT, windowing, server behavior, and framing are separate concerns. The parser remains correct even when the transport delivers bytes in unexpected chunk sizes.

Verify the result

- Are bits and bytes converted consistently?
- Does the client handle both fragmented and coalesced reads?
- Are message size and total waiting time bounded?

Extend the exercise

Add a request ID and simulate losing the response after server processing. Explain why reconnecting and retrying a write can duplicate a business action even though TCP itself avoids duplicate bytes within one connection.

Make the learning stick.

Does UDP mean packets always arrive faster?

No. UDP provides datagrams without TCP's connection and reliability machinery, but applications still face loss, congestion, security, and ordering needs. Protocol choice depends on the desired semantics and workload.

Why can ping work while an API fails?

ICMP reachability does not prove DNS correctness, TCP port access, TLS validity, proxy behavior, or application health. Test the same path and protocol as the failing request before drawing a conclusion.

Is a timeout proof that the server did nothing?

No. The operation may have completed and its response may be delayed or lost. Reads can often be retried safely; side-effecting requests need a defined identity, status lookup, or reconciliation policy.

Avoid these traps

- Assuming one socket read contains one complete application message.
- Confusing receiver flow control with network congestion control.
- Retrying every failed write with a fresh logical operation ID.

Ready for the next step?

- A browser request can be explained across protocol boundaries.
- Partial reads and incomplete messages are handled correctly.
- Timeouts are distinguished from known operation failure.
- Measurements identify latency components instead of guessing.

Reliable next reads

- [MDN: HTTP overview](#)
- [Beej's Guide to Network Programming](#)
- [RFC 9000: QUIC](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.